



Harness 101: 从 ReAct Loop 讲起

2026年4月20日的录屏在这里：

<https://larkcommunity.feishu.cn/minutes/obcn9e9991868drx14v22am7>

📖 Harness 101: 从 ReAct Loop 讲起 ➡️ 点击此处领取教程配套材料

1. 前言

现在打开任何一篇 Agent 相关的讨论，扑面而来的词都是 Multi-Agent、Deep Research、Skill、Sub-Agent、Orchestrator —— 一个比一个抽象，一个比一个唬人。但如果你把这些花哨东西的外壳一层层剥开，会发现最底下跑的其实是同一条 loop：**模型想一下，调个 Tool，看结果，再想一下** —— 一条最朴素的 ReAct Loop。

笔者最近在出差的路上，利用从上海到北京的火车时间，半 Vibe Coding 半古法的、从零做了一个开源的 Harness + Coding —— **Helixent**（现已由我和社区一起继续维护，欢迎 Star），用来给同事们讲解 Harness，试过好几种讲法，但是发现如果走来就看代码，沟通成本过高。从 Multi-Agent 倒推讲起，听众很快就迷失在调度图里；从 Claude Code 这种成熟产品切入，又容易被它的工程复杂度带偏。最后发现先不说代码，用 Playground 把 Harness 掰开来看本质，这种效果最好：

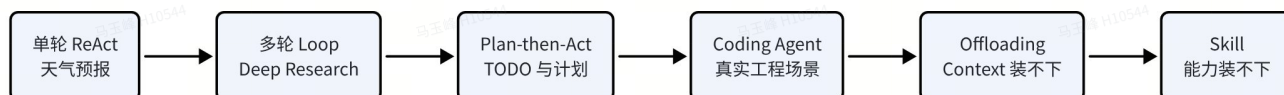
📖 从「天气预报」这个最土的例子正着推。让模型调一次 `get_weather`，再让它连调两次，再让它自己写 plan，再把它扔进 Coding Agent 的场景，最后让它面对装不下的上下文和塞不下的能力。每一步都只比上一步多一点点，但等走完五步，你再回头看 Deep Research、Claude Code、Skill，会发现它们都只是这一条线上的某个站点而已。

所以今天就带大家不写代码，通过 Playground 走马观花的深入浅出 Harness 中的各种概念。

你知道吗 —— ReAct 这个词从哪来？

ReAct 出自 Yao et al. 2022 年的论文 [ReAct: Synergizing Reasoning and Acting in Language Models](#)。它首次提出把 **Reasoning（推理）** 和 **Acting（行动）** 交织起来，让模型一边想一边调外部工具而不是一口气想完再做。论文在 ALFWorld 上相比基线绝对成功率提升 **34%**，在 WebShop 上提升 **10%**，且只用了一两条 in-context example。笔者从 2023 年 4 月开始做 ReAct Loop Agent，今天几乎所有 Agent Harness 跑的都是这条 loop 的变体。

这条演化阶梯大概长这样：



每一级往上加的东西都极少：多轮只是让 loop 多转几圈；Plan 只是在 loop 外头多塞一条 TODO；Coding Agent 只是把 Tool 从 `web_search` 换成 `read_file`、`bash`；Offloading 和 Skill 解决的则是「东西多到塞不进 Context Window」之后的两个不同方向的妥协。笔者相信，理解 harness 的关键不是记住每一级的名字，而是看清**每一级是为了解决上一级剩下的什么问题**。

延伸阅读

[📖 Agent Loop：从 for 循环到自治系统的进化之路](#)

本文走的就是这条路。我们从最简单的天气预报说起。

加入 Agentara 2 群：

**Agentara Underground 情报站（2群）** 外部

每日投喂 AI/Agent 新闻，不定期推送热点深度研...

群名片

加入 Agentara Underground 情报站

加群获取更多线上资料，和 7,500+ 位 AI 爱好者一起学习 Prompt、Context Engineering、Harness Engineering 和 Skills。

已经加入过 1 群的小伙伴不用重复加入，也请分享给需要的同学们

加入本节课专属讨论区：



Harness 101 公开课: 从 ReAct Loop 讲起

外部

群名片

Harness 教程

- [📖 Harness 101: 从 ReAct Loop 讲起](#) (适合非技术同学)
- [📖 Harness 101: Claude.AI 提示词与记忆结构解析](#) (适合非技术同学)
- [📖 Harness 101: Claude Code Agent 里的常用工具一览](#)
- [📖 Harness 101: Context Offloading 机制](#)
- [📖 Harness 101: 写给 Agent 的虚拟文件系统](#)

Skill 101 系列

- [📖 Skill 101: 前传](#) (适合非技术同学)
- [📖 Skill 101: 如何写好 Skill](#) (适合非技术同学)
- [📖 Skill 101: 并行多任务](#)
- [📖 Skill 101: 可以自动更新的 Skill](#) (适合非技术同学)

AIGC 系列

- [📖 用即梦 + GPT + Images 2.0 做 AI Vlog](#) (适合非技术同学)

开源项目

- [DeerFlow](#): 6.4 万颗  Star 的开源 Agent Harness
- [Agentara](#): 套壳 Claude Code 和 Codex 的 Claw-like App
 - 底层同时支持 Claude Code 和 Codex
 - 支持任务队列, 可同时跑 n 个并行任务
 - 飞书 IM 友好, 针对飞书优化了输出的卡片交互
 - 包含大量第三方及原创的 Skills, 包括健康医疗、AIGC skills 等
- [Helixent](#): 一个不依赖于任何框架的、从零开始写的 Agent Harnesss

- 一个精巧而完整的、原创 Harness，包含了从 Model 到 Agent Loop，再到 Coding Agent 的全部实现
- 附带一个相对精美的 CLI 界面

🌐 社交网络

- [我的小红书](#)
- [我的 Github](#)

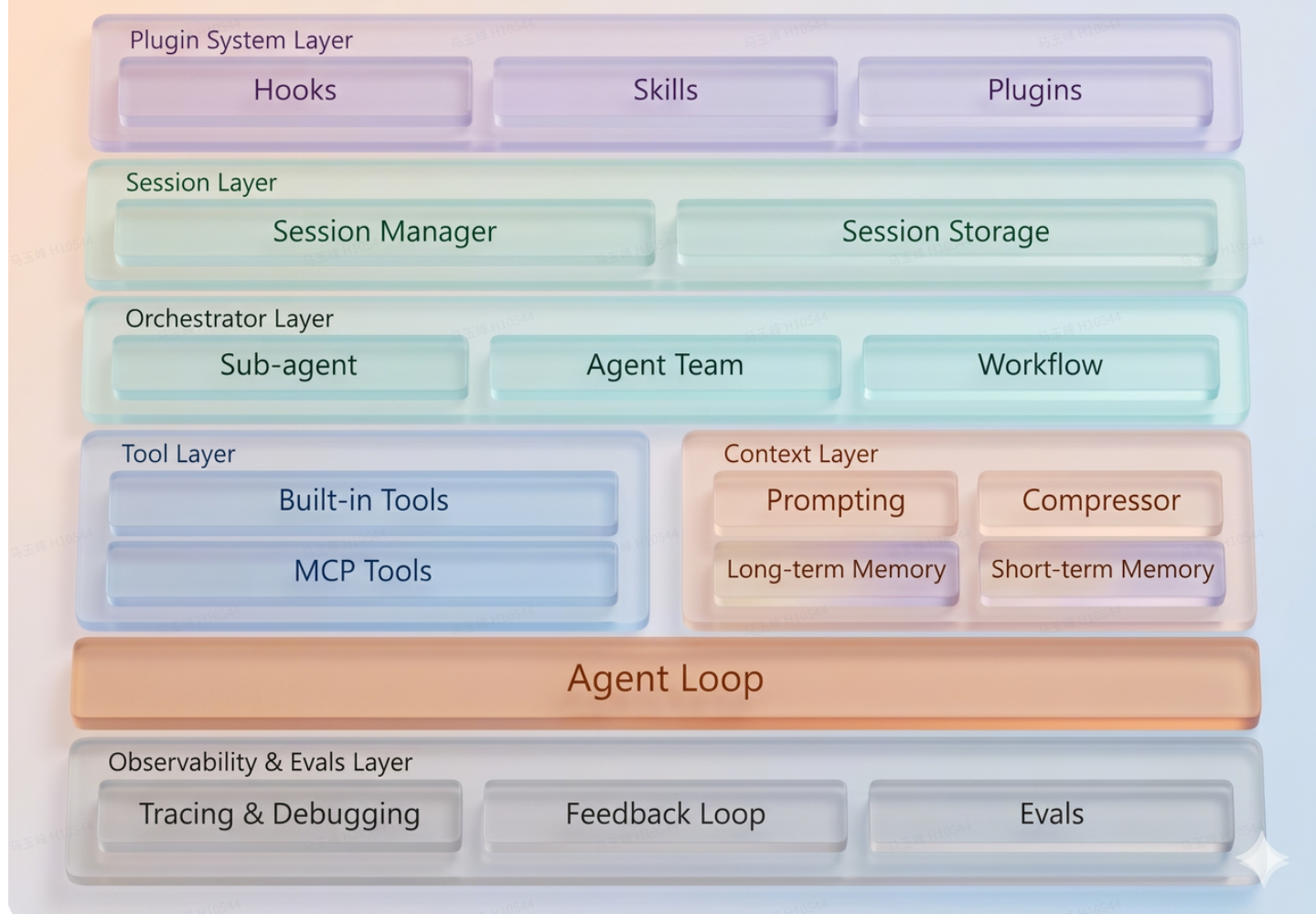
💰 赛博化缘

- 觉得我的课程或者文章对你有启发？你的鼓励，是我持续打磨课程、产出更多干货的动力。
- [点击这里](#) 💰 打赏： [👉 为爱发电](#)

2. 透过现象看本质

2.1 Harness 的本体

Harness 101: Overview



“对于什么是 Harness，世界上没有两个人的答案会是一样的”。如上图（由 Nano Banana Pro 2 生成）所示，在我看来 Harness 必须包含的是 **Agent Loop**、**Tool Layer**、**Context Layer**，而最重要的毫无疑问是 **Agent Loop**，也就是本文的主角。在今后的 **Harness 101** 系列课程，我们还将会无数次回到这张 Overview 架构图，把它拆开了、揉碎了、又装回去的讲清楚。

- **Agent Loop**: Harness 的运行核心，驱动 LLM ↔ Tool ↔ Context 的循环调度，每一轮都要决定“调工具还是出终态”。
- **Tool Layer**: Agent 在每一步 Loop 里可调用的能力集合。
 - **Built-in Tools**: 内置原子能力，如 `web_fetch`、`web_search`、`bash`、`read_file`、`write_file`、`edit`、`glob`、`grep`、`write_todos`、`agent`、`task_monitor`、`browser-use` 等。内置工具均可被替换和插拔。
 - **MCP Tools**: 通过 MCP 协议接入的第三方外部工具，运行时可插拔。
- **Context Layer**: 每轮 LLM 调用前的上下文装配与治理，决定“模型这一次看到什么”。
 - **Prompting**: 用 XML 把 System Prompt 拆成零件化组装，并在运行中注入 `<system-reminder>` 等 nudges。
 - **Compressor**: 对超长 trajectory 做压缩/摘要，守住 token 预算与注意力。

- **Short-term Memory**: 当前任务范围内的临时记忆 (scratchpad、todos、近期工具结果等)。
- **Long-term Memory**: 跨会话持久化的长期记忆 (用户画像、偏好、领域知识)。
- **Session Layer**: 面向用户界面的会话层 (类比聊天软件左侧边栏的会话列表)。
 - **Session Manager**: 会话的创建、切换、归档等生命周期管理。
 - **Session Storage**: 会话元数据与消息历史的持久化存储, 支持扩展。
- **Orchestrator Layer**: Multi-Agent 的编排调度层。包括:
 - **Workflow 架构**: 基于工作流式的编排, 类似 LangGraph、Coze、N8N 等。
 - **Sub-agent 架构**: 类似 Claude Code 的 Sub-agent 风格, 每一个并行执行的 Sub-agent 上下文隔离且互不通讯, 实现起来简易且高效, 常见于 Deep Research 和 Coding 任务。
 - **Agent Team 架构**: Claude Code 提出并实践的 Agent 团队架构, 每一个 Agent 都具有一个收件箱, Agent 在执行任务过程中可以异步通讯。
 - 其他编排架构和风格等
- **Plugin System Layer**: Harness 对外的可扩展接口, 让第三方在不改核心的前提下介入。
 - **Hooks**: 在 Agent Loop 的关键节点 (pre/post tool call、turn start/end 等) 注入自定义逻辑。
 - **Skills**: 按需加载的能力包, 走 SKILL.md 渐进式披露模式。
 - **Plugins**: 更粗粒度的扩展打包形式, 其实就是一组 Hooks/Skills/Tools 而已, 可以通过 `{plugin_name}:{skill_name}` 的形式来调用。
- **Observability & Evals Layer**: Self-evolving Harness 的闭环底座, 让上面的所有层都"有据可依地进化"。
 - **Tracing & Debugging**: 记录每次运行的完整 trajectory, 支持回放、归因与线上排障。
 - **Feedback Loop**: 把线上信号 (用户反馈、失败用例、Evals 退化项) 回流到 Prompt、Skills、Hooks 的迭代。
 - **Evals**: 离线基准与回归测试, 对 Harness 任何一次改动做数据化验收。

Self-evolving 闭环: Tracing 采集 trajectory → Evals 打分识别短板 → Feedback Loop 把短板回流到 Prompting/Skills/Hooks → 上层持续迭代, Agent Loop 在下一轮就跑在更好的 Harness 上。

接下来, 让我们来看一段用于通过 Harness 创建 Agent 伪代码:

代码块

```

1  # 创建一个 Sub-agent 风格 Multi-Agent 架构的 Lead Agent
2  lead_agent = create_agent_loop(
3      prompt, # System prompt
4      tools=[*builtin_tools, *mcp_tools, *customized_tools], # 工具箱
5      sub_agents=[general_purpose_agent, code_rag_agent], # 子 Agents, 且子
        Agents 通常也是由 create_agent_loop() 创建
6      hooks=hooks, # 各类 Hooks 插件
7  )

```

没错，Harness 的核心是 Agent Loop，而一个 Agent Loop 最起码由以下几个核心部分组成：

1. **System Prompt** 定义 Agent 的身份、行为边界、输出格式、思考方式和记忆，是整个 Loop 的“灵魂”，也是 Context Engineering 最核心的战场。你可能听说过 AGENTS.md、SOUL.md 和 IDENTITY.md 等等，本质上他们都被静态或动态加载到 System Prompt 里的。
2. **Tools** Agent 能调用的能力集合。没有 Tools，Agent 只是一个会说话的 LLM，有了 Tools 才能真正“做事”。包括：
 - `builtin_tools`：文件读写、代码执行、搜索等
 - `mcp_tools`：外部 MCP 服务集成
 - `customized_tools`：业务定制能力
3. **Loop 控制逻辑** 决定“何时继续、何时停止”的机制
4. **Hooks** 插入 Loop 生命周期各节点的钩子，这才是 Harness 区别于裸 Agent 的核心价值，包括 Skills、TODO、Memory、权限审批等等都可以通过 Hooks 实现，LangGraph 称之为 Middleware

2.2 Agent Loop 的本体

然而实际上 Agent Loop 做的事情少得令人失望。核心就是下面这段循环：

代码块

```

1  messages = [{"role": "user", "content": user_input}]
2  hooks.before_agent(context)
3  iterations = 0
4  while True:
5      iterations = iterations + 1
6      hooks.before_model(context)
7      response = llm.call(messages, tools=TOOLS)
8      hooks.after_model(context)
9      messages.append(response)
10     if not response.tool_calls or iterations > MAX_ITERATIONS:
11         # 模型不再调工具，loop 终止
12         break
13     # 并行执行所有 tool_call，把结果打包回去

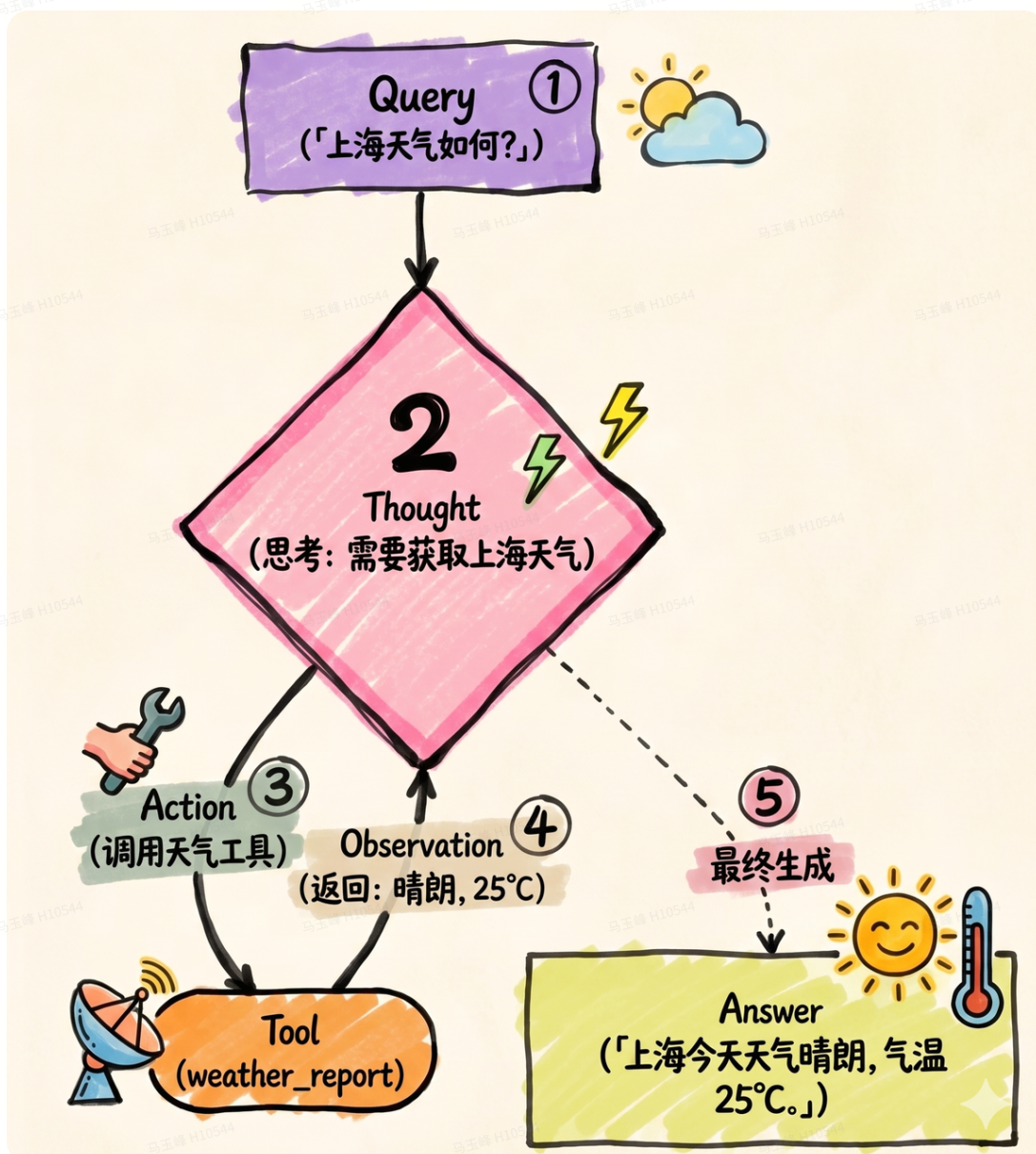
```



```
14     hooks.before_tool(context, response.tool_calls)
15     results = run_tools_parallel(response.tool_calls)
16     hooks.after_tool(context, response.tool_calls, results)
17     messages.extend(results)
18     hooks.after_agent(context)
19     return response.content
```

👊 坚持住这是本文最后一次出现代码了！本次第一讲不需要你掌握编程技术。

3. 单轮 ReAct Loop



3.1 天气预报

设想一个最小场景：用户问「北京今天冷吗」。我们给模型一把 Tool：

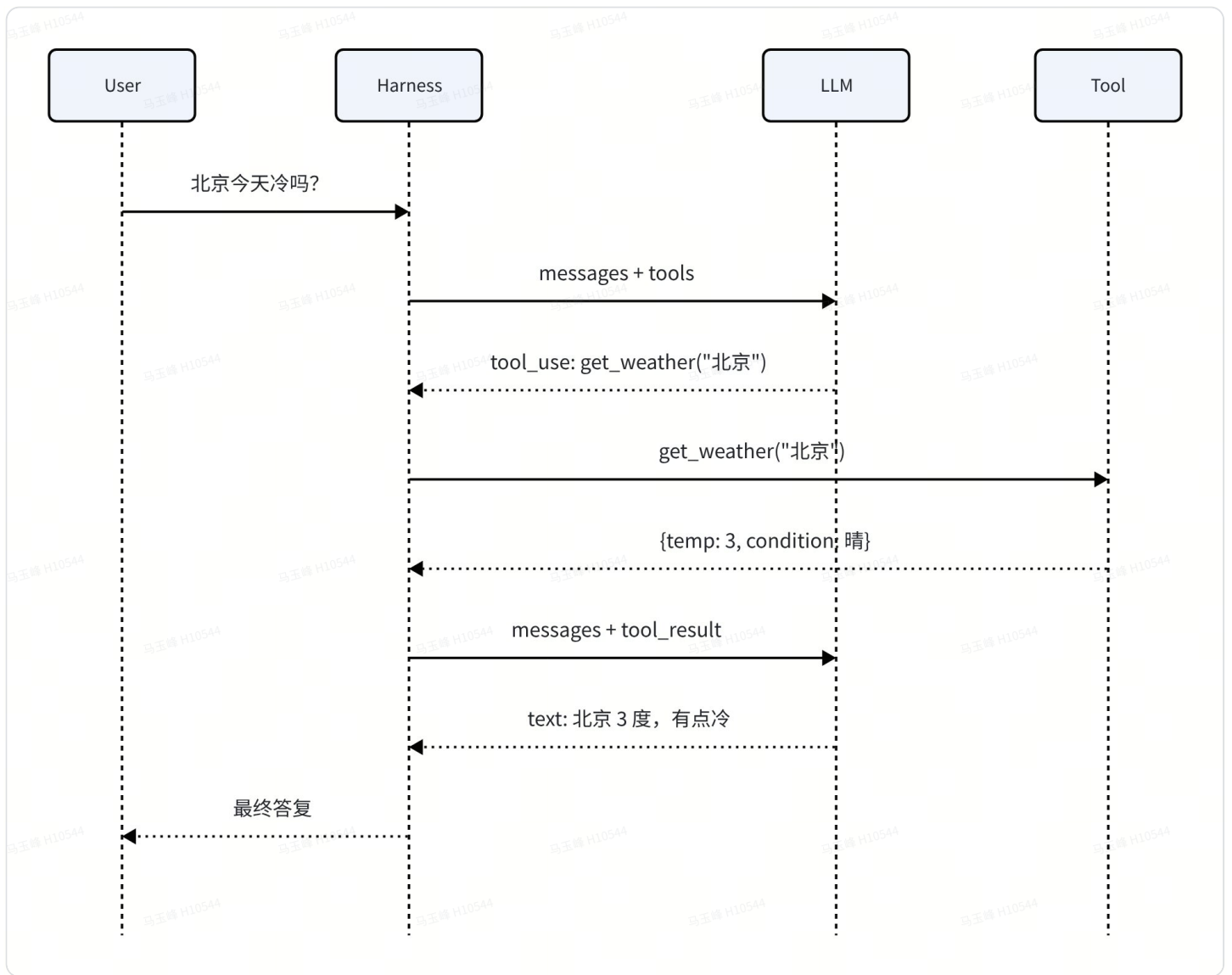
代码块

```
1  {
2    "name": "weather_report",
3    "description": "Retrieves a weather report for a specified city. Use this
when you need current weather conditions or a concise weather summary for a
known city name. Do not use this when geographic coordinates, historical
weather data, weather alerts, or multi-day forecasts are required unless those
capabilities are explicitly supported elsewhere.",
4    "strict": true,
5    "parameters": {
6      "type": "object",
7      "required": ["city"],
8      "properties": {
9        "city": {
10         "type": "string",
11         "description": "Use the Chinese name of the city for which to retrieve
the weather report. Do not use city code."
12       }
13     },
14    "additionalProperties": false
15  }
16 }
17
```

模型拿到用户的问题之后会做一件事：它判断「光靠自己答不了，得查一下」，于是 emit 一个 `tool_use`，内容是 `get_weather(city="北京")`。Harness 拿到这个 `tool_use`，去真的执行函数，把结果（比如 `{"temp": 3, "condition": "晴"}`）作为 `tool_result` 塞回消息队列。然后模型再看一眼结果，决定是再调一次 Tool 还是直接回答用户。如果直接回答，它 emit 的就是一段纯文本，没有 `tool_use` —— loop 结束。

这就是 ReAct 的一个完整回合：**Reason（决定要调什么）→ Act（发出 tool_call）→ Observe（看 tool_result）**，然后要么再来一轮，要么给出最终答复。

Harness 不做决策。它不判断「搜得够不够了」「该不该换个关键词」，它唯一做的判断是：**这一轮 assistant turn 里有没有 tool_call？有就执行，没有就退出**。所有的「想」都交给模型，harness 只负责搬运——把 `tool_call` 喂给工具，把 `tool_result` 喂给模型。此外，还有一堆 Hooks 用于扩展 Harness 能力，同时也可以用来做 Tracing。



3.2 一次 emit 多个 tool_call

真正有意思的是下一个问题：「北京和上海哪个更冷？」

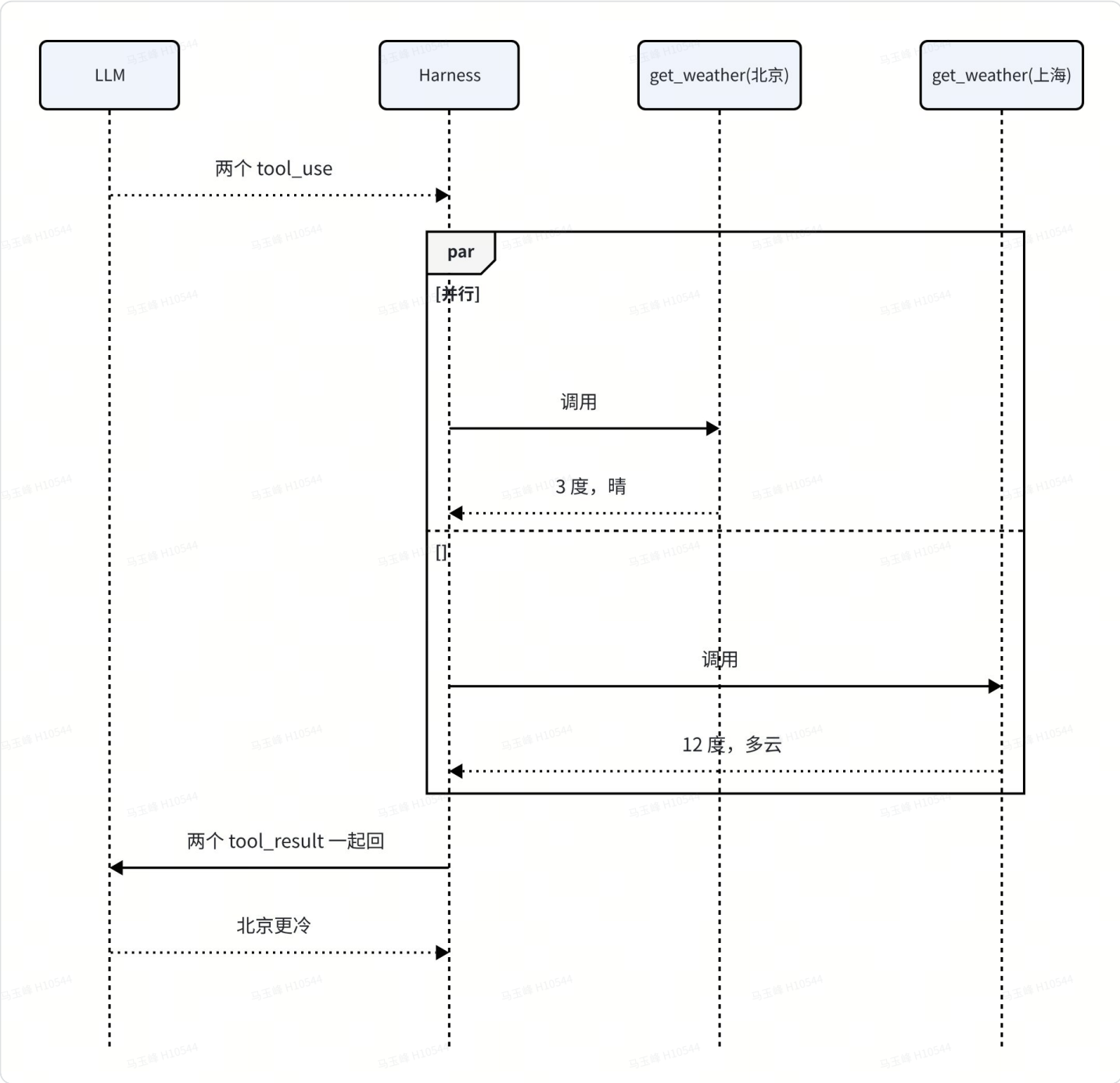
一个称职的模型不会傻乎乎地先查北京再查上海，而是在**同一个 assistant turn 里 emit 两个 tool_use**：

代码块

```
1  {
2    "role": "assistant",
3    "content": [
4      {"type": "tool_use", "id": "t1", "name": "get_weather", "input": {"city":
5        "北京"}},
6      {"type": "tool_use", "id": "t2", "name": "get_weather", "input": {"city":
7        "上海"}}
8    ]
9  }
```

Harness 这一侧只要把 `run_tools` 写成并行执行（上面伪代码里的 `run_tools_parallel`），两次查询就能同时跑，拿到两个 `tool_result` 再一起塞回去。模型下一轮看到两个结果并排摆着，就能直接做对比回答。

这是 ReAct 一个常被低估的性质：一个 **assistant turn** 可以对应任意多个 **tool_call**。你写 harness 时千万别默认「一次一个」，否则一遇到「对比三座城市」「并行读十个文件」这种场景就会慢得离谱。



3.3 单轮够用吗

天气预报这个例子之所以适合打底，是因为它把 ReAct 的骨架露得干干净净：**Tool schema**、**assistant turn**、**tool_use**、**tool_result**、**停机 (halt) 条件**，一共就这几件东西，少一件都跑不起来，多一件都是后话。

但你一旦稍微改一下问题——「这周五北京下雨概率大吗？对比一下过去五年同期的降雨量。」——单轮就不够了。模型得先查这周五的预报、再查历史数据、再对比、可能还要再查一次补齐某个缺失的年份。它需要**多轮** loop 来回踱步，每一步都依赖上一步的结果来决定下一步查什么。

这就是我们要去的下一站。

4. 多轮 Loop: Deep Research

上一章的单轮够天气预报，但遇到「这周五北京下雨概率对比过去五年」这种题就要跨轮次查了。把 `get_weather` 换成两把更通用的 Tool，就得到了 Deep Research 的雏形：

代码块

```
1  {
2    "name": "web_fetch",
3    "description": "Fetches and returns content from a specific webpage or URL.
    Use when you already know the exact URL and need to retrieve its contents for
    reading, extraction, or analysis. Do not use when you need to discover
    relevant pages first; use a search function instead.",
4    "strict": true,
5    "parameters": {
6      "type": "object",
7      "required": ["url"],
8      "properties": {
9        "url": {
10         "type": "string",
11         "description": "The full URL of the webpage or resource to fetch"
12       },
13       "timeout_seconds": {
14         "type": "integer",
15         "description": "The maximum number of seconds to wait for the fetch
    request to complete"
16       },
17       "follow_redirects": {
18         "type": "boolean",
19         "description": "Whether to follow HTTP redirects when fetching the URL"
20       }
21     },
22    "additionalProperties": false
23  }
24 }
25
```


代码块

```
1  {
2    "name": "web_search",
3    "description": "Searches the web for relevant pages, sources, or recent
information based on a query. Use when you need to discover information, find
candidate sources, or locate webpages related to a topic. Do not use when you
already have a specific URL to retrieve; use a fetch function instead.",
4    "strict": true,
5    "parameters": {
6      "type": "object",
7      "required": ["query"],
8      "properties": {
9        "query": {
10         "type": "string",
11         "description": "The search query to submit to the web search engine"
12       },
13       "max_results": {
14         "type": "integer",
15         "description": "The maximum number of search results to return"
16       },
17       "language": {
18         "type": "string",
19         "description": "The preferred language for search results, such as
'en'"
20     }
21   },
22   "additionalProperties": false
23 }
24 }
```

工具只有两把，但组合出的行为空间一下子大了几个数量级。模型可以 search 拿到一堆候选链接，挑一两条最相关的 fetch 下来读全文，发现信息不够再换个关键词 search，继续挑链接 fetch —— 整个过程是一条**走走停停、拐弯抹角**的轨迹，走多少步、怎么拐，全靠模型自己判断。

4.1 一段典型的 transcript

用户问：「Claude Code 的 Skill 机制是怎么工作的？」一次典型的多轮 loop 可能长这样：

代码块

```
1  # turn 1 - user
2  "Claude Code 的 Skill 机制是怎么工作的？"
3
4  # turn 2 - assistant
```

```

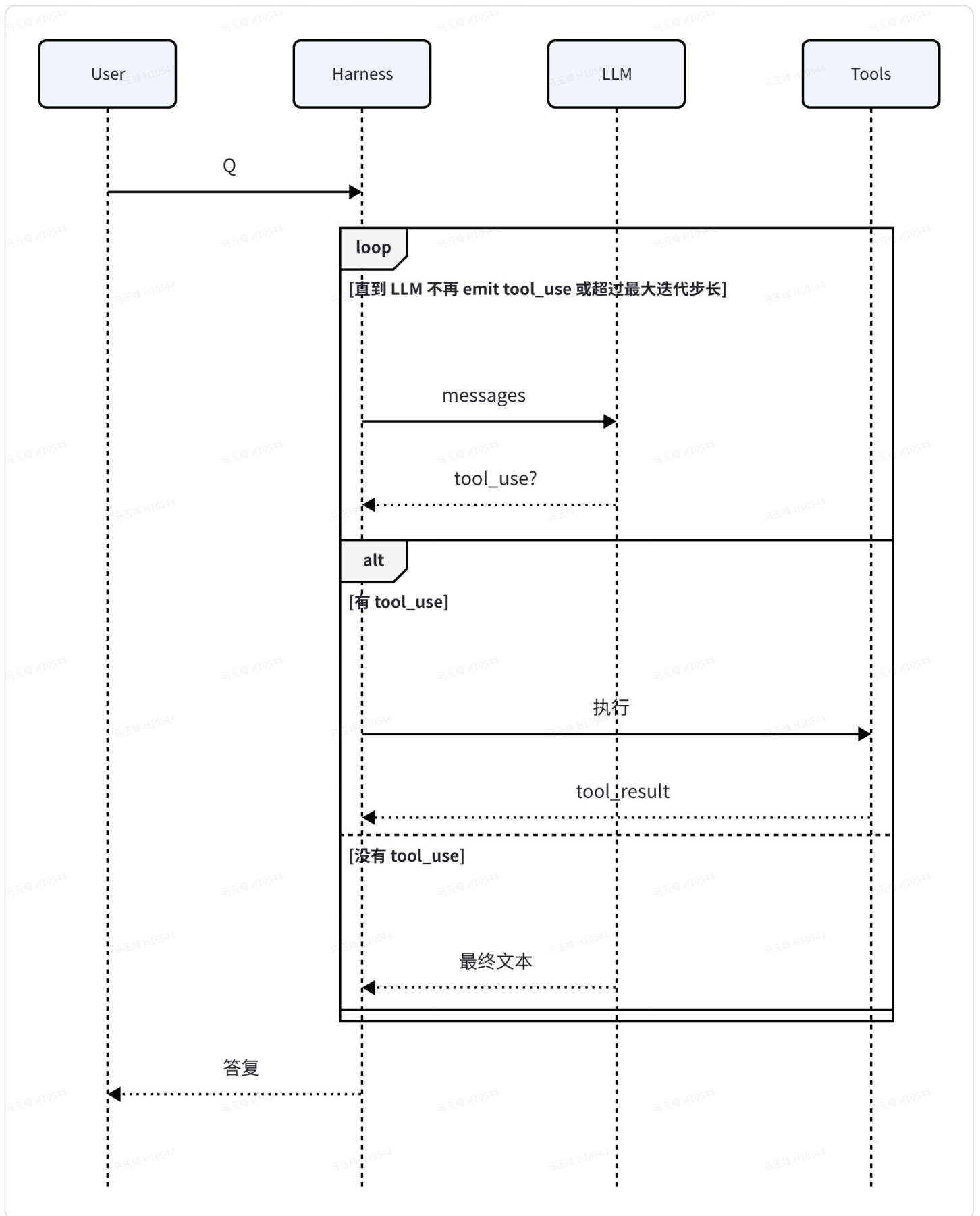
5  tool_use: web_search(query="Claude Code Skill 机制")
6
7  # turn 3 - tool_result
8  [ { title: "Introducing Claude Skills", url: "https://...", snippet: "..."},
9    { title: "How Skills work in Claude Code", url: "https://...", snippet:
10      "..."},
11    ... 8 more ]
12
13 # turn 4 - assistant
14 tool_use: web_fetch(url="https://.../introducing-claude-skills")
15
16 # turn 5 - tool_result
17 "... Claude Skills are composable capabilities packaged as folders containing
18 a SKILL.md ..."
19
20 # turn 6 - assistant
21 # 觉得还缺一层"触发机制"的细节
22 tool_use: web_search(query="Claude Code Skill progressive disclosure
23 description trigger")
24
25 # turn 7 - tool_result
26 [ ... ]
27
28 # turn 8 - assistant
29 tool_use: web_fetch(url="https://.../how-skills-work")
30
31 # turn 9 - tool_result
32 "... Skills are loaded lazily via a progressive-disclosure mechanism: ..."
33
34 # turn 10 - assistant
35 # 信息够了, 不再 emit tool_call, 直接回答
36 "Claude Code 的 Skill 是一个放在 .claude/skills/ 下的文件夹 ..."

```

注意这条轨迹的几个特点：每一步的 tool_call 都依赖前一步的结果（第二次 search 的关键词是读了第一篇文章之后才想出来的），search 和 fetch 交错出现，轨迹的长度事先不可知——可能三轮就收敛，也可能十五轮还没够。

4.2 多轮 loop 的形状

把这个过程画成时序图，形状就很干净：



Harness 的代码没变 —— 还是第二章那十行。变的只是 loop 会多转几圈。

4.3 谁决定 loop 什么时候停

这是多轮 ReAct 最关键也最反直觉的一点：**决定 loop 停机的不是 harness，而是 LLM 自己。**

一个新手写 harness 常犯的错误，是在 harness 层手动判断「嗯，搜了三次差不多该收尾了」「已经 fetch 过五篇文章，够多了」。这套做法本质上是把 workflow / DAG 那一套搬过来：每一步预先规划好，harness 负责推进。但 ReAct 不是这么玩的——ReAct 的 harness 只看一个信号：

这一轮 assistant turn 里，有没有 tool_use?

- 有——执行工具，把结果塞回 messages，再来一轮
- 没有——这就是最终答复，退出 loop

是否继续是**模型主动决定**的。它觉得信息不够，就继续 emit `web_search`；它觉得够了，就不 emit 任何 tool_use，直接输出文本。harness 不插手这个判断。

你知道吗——ReAct 和 Workflow Agent 最本质的区别就是谁决定停机？

在 LangGraph 这类 workflow / DAG 式 agent 里，节点之间的跳转由你预先画好的图决定，是否结束由图的终点决定——这是**外部调度**。而 ReAct loop 里谁也没画图，是否继续完全取决于模型这一轮有没有吐出 `tool_use`——这是**主动停机**。这个差别看似小，但它决定了 ReAct 天然能处理「事先不知道要走几步」的开放式任务（Deep Research、Coding），而 workflow agent 更擅长边界清晰的流水线任务。

4.4 还差什么

多轮 loop 把「一步不够用」这个问题解掉了。但它有一个肉眼可见的毛病：**模型想到哪走到哪**。它没有全局视野，容易走偏——比如一个复杂问题本该从五个维度都调研一遍，它可能沉迷在第一个维度里搜了七八次，后面几个维度全忘了。或者它一开始就选了一个错误的关键词，后续几轮都在错误的方向上打转，浪费 Token 又得不到答案。

换句话说，**纯 ReAct 的 loop 只有战术，没有战略**。下一章我们要给它加上战略层。

5. Plan-then-Act：给 Loop 加一层战略

纯 ReAct 多轮跑复杂问题时有个结构性毛病——模型是「走一步看一步」，没有全局视野。碰到一个需要从五、六个维度拼凑答案的调研题，它会在前三步里顺手挑几个最显眼的角度查一查，然后就觉得「差不多了」给你收尾。漏掉的那几个维度它自己也不知道漏了。

怎么治？一个粗暴而有效的思路——**让它先把 plan 写下来，再去执行**。

5.1 一个新工具：`write_todos`

在 Deep Research 的 harness 里，我们给 Agent 加了一件新工具 `write_todos`。它什么都不做，就是把一个 TODO 列表写到自己的 state 里。System prompt 里那一句就是开关：

代码块

```
1  <agent>
2  You're a helpful assistant that can utilize the tools provided to answer
   questions and help with tasks.
3  </agent>
4
5  <behaviors>
6  <deep-research activate-when="User explicitly asks to research, investigate,
   dig into, or study a topic deeply; OR when the user's query involves a complex
   or unfamiliar topic that would benefit from structured investigation">
7  - ⚠ CRITICAL: You MUST follow this workflow when activated. Skipping it is a
   violation.
8  - Step 1: Immediately call `web_search` once for initial orientation on the
   topic (do NOT skip this).
9  - Step 2: Write a research plan via `write_todos` with 4-8 items that expand
   both breadth and depth.
10 - Step 3: Execute each TODO item sequentially or in parallel. After each item,
   mark it completed.
11 - Step 4: Dynamically adjust the plan based on new findings.
12 - ⚠ Do NOT jump to a final answer before completing the research plan.
13 </deep-research>
14 </behaviors>
```



你知道吗？为什么要用 XML 来写 Prompt

熟悉 Anthropic 提示词工程的朋友们一定都发现了，从 2023 年起，它们家使用的就一直是 XML 风格的 Prompt，2025 年底在 OpenAI GPT 5 的示例中，最外层也从 Markdown 变成了 XML 里封装 Markdown 了。这是因为与 Markdown 比，XML “更容易”让模型找到开闭区间。同时，在拼装 Prompt 时，也更加区块化、组件化（甚至有人为此还开发了框架），这就要求编写 Prompt 的人始终记住要将某一个独立的功能封装在 XML 标签之间，以方便通过外层传入的 Flags 进行动态增减。

注意看 `<agent>` 里的 `Use write_todos to make a plan and break down tasks.`——这一句话就让模型的行为模式从「想到哪走到哪」切换到了「先 plan 再 act」。

工具本身的 schema 非常朴素：

代码块

```
1  {
2    "name": "write_todos",
```



```

3    "description": "Creates or updates TODOs for complex, multi-step tasks. Use
    this when a task benefits from being broken into several actionable items that
    can be tracked or revised over time, for example deep research or coding. Do
    not use this for very simple tasks that only require one or two steps. Should
    call this tool after each TODO is done. Mark the first item as 'in_progress'
    if applicable.",
4    "strict": true,
5    "parameters": {
6        "type": "object",
7        "required": ["todos"],
8        "properties": {
9            "todos": {
10               "type": "array",
11               "description": "A list of TODO items representing the steps or sub-
tasks needed to complete a complex task.",
12               "items": {
13                   "type": "object",
14                   "required": ["content"],
15                   "properties": {
16                       "content": {
17                           "type": "string",
18                           "description": "The text description of the TODO item."
19                       },
20                       "status": {
21                           "type": "string",
22                           "description": "The current status of the TODO item, such as
pending, in_progress, or completed."
23                       },
24                       "priority": {
25                           "type": "string",
26                           "description": "The priority level of the TODO item, such as
low, medium, or high."
27                       }
28                   },
29                   "additionalProperties": false
30               }
31           },
32           "additionalProperties": false
33       }
34   }
35 }

```

一个 TODOs 看上去是这样的：

代码块

```

1  {
2    "todos": [
3      {
4        "content": "搜索张雪机车的基本信息，确认品牌背景和产品线",
5        "status": "completed",
6        "priority": "high"
7      },
8      {
9        "content": "查找张雪机车走红的时间节点和关键事件",
10       "status": "completed",
11       "priority": "high"
12     },
13     {
14       "content": "分析产品特点：性能、设计、价格等",
15       "status": "in_progress",
16       "priority": "medium"
17     },
18     {
19       "content": "研究营销策略：社交媒体、网红推广等",
20       "status": "pending",
21       "priority": "medium"
22     },
23     {
24       "content": "收集用户评价和市场反馈",
25       "status": "pending",
26       "priority": "medium"
27     },
28     {
29       "content": "综合分析火爆原因，形成结论",
30       "status": "pending",
31       "priority": "high"
32     }
33   ]
34 }

```

5.2 模型的新行为

加上这一件工具，原来的 Loop 立刻变了形状。拿上一章那个问题——「帮我调研一下 2026 年中国新能源车出海欧洲的现状」——接下来的 transcript 大致是这样：

代码块

```

1  user: 帮我调研 2026 年中国新能源车出海欧洲的现状
2  assistant → tool_use: write_todos([
3    { content: "了解 2026 年中国新能源车出口欧洲的整体规模", status: "in_progress" },
4    { content: "梳理主要玩家及其在欧份额", status: "pending" },

```

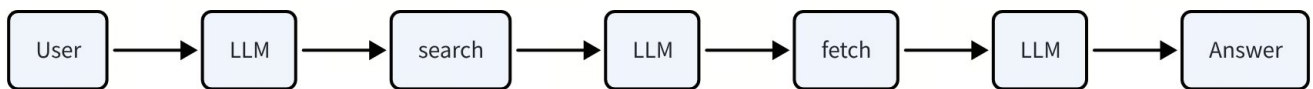
```

5     { content: "了解欧盟关税政策与反补贴调查进展", status: "pending" },
6     { content: "总结本地化建厂与渠道布局", status: "pending" }
7   ])
8   assistant → tool_use: web_search("2026 中国新能源车 欧洲 出口 数据")
9   ... (web_search / web_fetch 若干轮) ...
10  assistant → tool_use: write_todos([
11    { content: "了解 2026 年中国新能源车出口欧洲的整体规模", status: "completed" },
12    { content: "梳理主要玩家及其在欧份额", status: "in_progress" },
13    ...
14  ])
15  ... (继续执行剩余项) ...
16  assistant: <final answer>

```

和纯 ReAct 多轮相比，最大的差别——开头那一下 `write_todos`，把答题的「考纲」预先写死了。执行时漏掉一个维度，在 plan 上就会留下一个 `pending` 没划掉，模型自己会回过头来补。

画成图对比一下。先看纯 ReAct 多轮：



每一步只看当下的 `tool_result`，不回头也不前瞻。再看 Plan-then-Act：



多出来的是开头的 **plan** 和**每一步之后回头更新状态**。Loop 没变，只是多了一件工具、多了一条 `system prompt` 的指令。



你知道吗——为什么 `write_todos` 是工具，不是 `prompt` 里的结构化字段？

这个设计挺微妙。最朴素的做法是在 `system prompt` 里加一段「请先输出一个 `<plan>...` `</plan>` 块，再开始行动」——但这种 `plan` 只是一段文本，模型在后面的回合里可以装作没看见。把 `write_todos` 做成**工具**之后，`plan` 的产出变成了一次真正的 `tool_use` / `tool_result` 往返，会被原封不动地保留在 `context` 里；更关键的是，模型对自己「调用过的工具」有非常强的「已承诺」感——这是 RLHF 训出来的结构性倾向。工具调用在它眼里是动作，`prompt` 里的文本是建议。这两者的份量不一样。

到这里，Plan-then-Act 的骨架就搭完了。但实战中你很快会撞见两个小坑：

一是不熟悉的话题上，**plan 会跑偏**——模型对这个领域本来就一知半解，凭着半吊子理解列出的 4 个子任务，可能从一开始就漏了最关键的那一维。

二是跑到一半，**模型会忘了更新 todos**——明明做完了第二项，却不调 `write_todos` 把它划成 `completed`，也不把刚发现的一个新子任务补进去。Plan 就这么一点点失效了。

下面两节，分别打两个小补丁。

5.3 补丁 A：Plan 之前先做一次 briefing

Plan-then-Act 的第一个坑很好理解——**模型不能 plan 自己不认识的东**西。

假设用户抛过来一个冷门术语，比如「帮我调研一下 pp-ocr 在长尾中文古籍场景下的表现」。模型心里对 pp-ocr 只有一个模糊印象（「好像是百度开源的 OCR 引擎？」），直接 `write_todos` 的话，大概率会列出四条不咸不淡的 plan：查一下是什么、查一下性能、查一下优缺点、总结。真正要紧的维度——它在多语言上的分支、历史版本号、和某个竞品的 benchmark——一个都没踩到。

补丁也很轻。在 system prompt 里明确允许 Agent 在 plan 之前做一次 `web_search`，只为建立对这个话题的基本认知：

代码块

```
1 <guidelines>
2 <guideline for="deep-research">
3   - Create a research plan using `write_todos`
4   - Perform a quick `web_search` before planning if you're not familiar with the
      topic, or the topic may happen after the cut-off date
5   - Execute the plan step by step or in parallel if applicable
6   - Dynamically change the TODO plan according to the latest information we
      collect
7 </guideline>
8 </guidelines>
```

注意这句话里的两个关键词：`one-time` 和 `before planning`。它不是给模型一条「搜索随便使用」的空白支票，而是一条硬编码到 system prompt 里的**协议**——在 plan 之前，你可以也只用一次 `web_search` 来给自己做 briefing。

有了这一次 briefing，模型会先搜一把「pp-ocr」、看完前几条结果摘要，知道了它是 PaddlePaddle 体系里的 OCR 模型族，有 v1 到 v5、中英日韩多语种分支、服务器端和轻量端两条路线……**带着这份认知**再 `write_todos`，列出的 plan 就会从「查一下是什么」这种无效维度，变成「对比 v4 vs v5 在手写古籍上的识别率」这种有抓手的子任务。



你知道吗——为什么限制「仅一次」？

如果允许 Agent 在 plan 之前无限搜，它会一直搜下去——搜到感觉「差不多了」再开始 plan。那这个 harness 就退化成了纯 ReAct 多轮，plan 这件事形同虚设：所有信息都是在

plan 之前攒齐的，plan 只是把已经知道的东西抄一遍。把额度压到「仅一次」，相当于告诉模型：你对这个话题有一次建立语感的机会，别浪费——用完之后就必须坐下来好好列 plan、再按 plan 去深挖。一次刚好卡在「让 briefing 有意义」和「不抢 plan 主场」的平衡点上。

打完这个补丁，plan 的输入侧就稳了。下一个坑在输出侧——模型跑着跑着容易忘了回头更新 todos。

5.4 补丁 B：每一步之后 nudge 一下

第二个坑——模型会忘记维护 todos。

这个现象你肯定见过：开头 plan 列得漂漂亮亮 4 项，第一项做完了，模型就急着开始做第二项；做到第三项的时候，发现需要补一个新的子任务，但它直接就搜了，没把这个新发现追加进 todo list。跑完全程回头看 plan，状态还停在开头那份——第一项之后就再也没更新过。

对这个问题，仅仅在 system prompt 里写一句「记得更新 todos」是不够的。prompt 是一次性铺好的背景板，模型读完前几轮之后，它的注意力权重已经转移到更近处的 tool_result 上了，背景板上的叮嘱会被逐渐淡化。

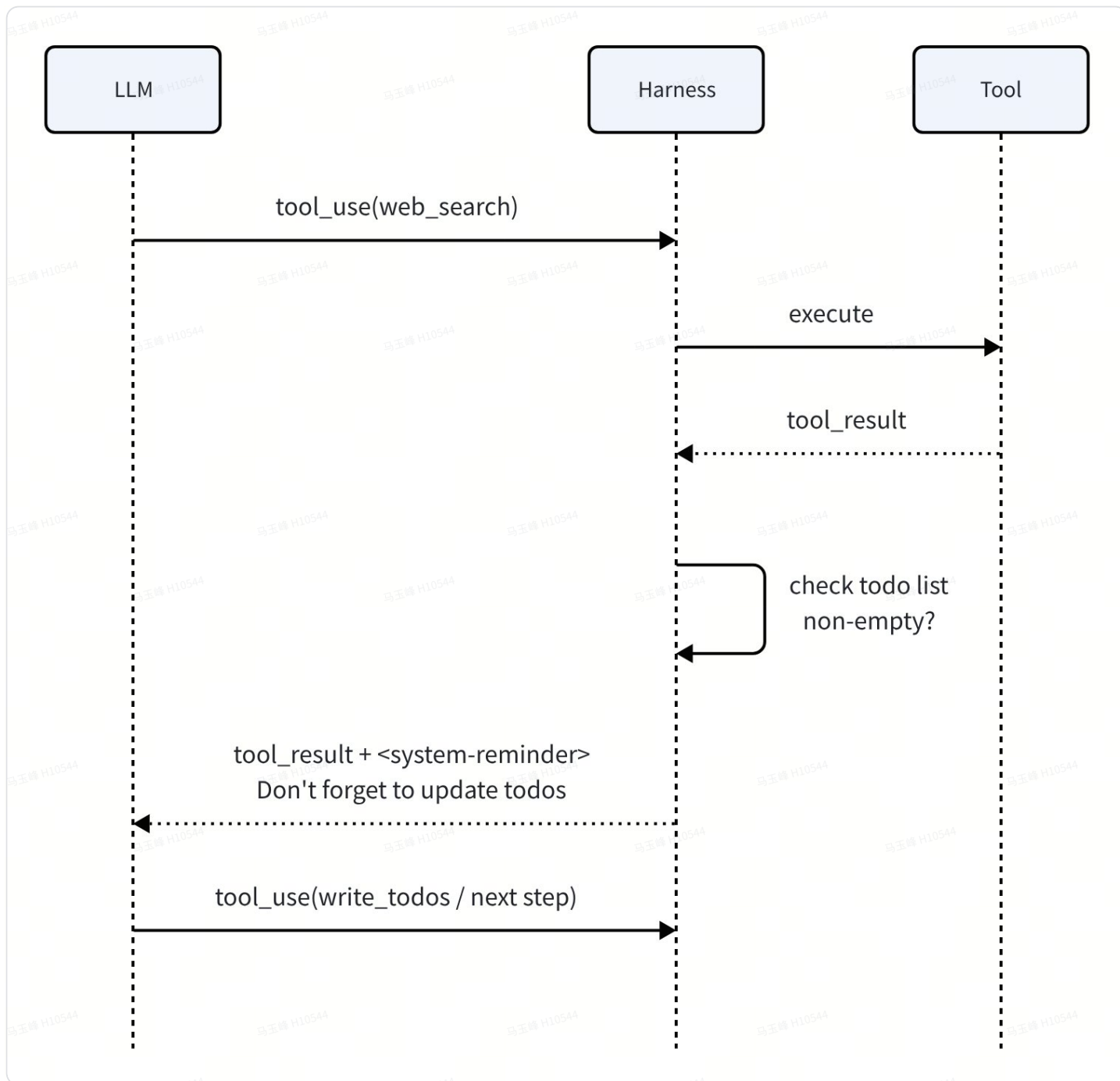
Harness 的解法叫 **nudge（提醒 / 推一把）**，也叫 **system-reminder**——在每次最后一次 **tool_result** 之后、且当前 todo list 非空时，由 harness 硬编码往上下文里追加一条提醒给模型：

代码块

```
1 <system-reminder>
2 - There're still some TODOs not marked as 'completed', update the TODO list
  before your next action
3 - Running multiple tasks in parallel is allowed, and mark multiple TODOs as
  'in_progress' if applicable
4 - Add/update/remove TODOs from your plan according to the latest information
  we collect if necessary
5 <system-reminder>
```

这条提醒不是用户发来的 message，也不是模型自己生成的 assistant message，而是由 harness 在 tool_result 之后「插播」的一条 system-reminder。从模型视角看，它每做完一件事、正准备盘算下一步时，都会看到这一句在眼前。

画成时序图更清楚：



注意 Harness 的两个动作：一是把 `tool_result` 原样回传给 LLM；二是在检查到「当前 todo list 非空」时，**额外**追加一条 system-reminder。两件事在同一个消息里交给 LLM，看起来像是「工具返回自带了一句 nudge」。

关键是——这条 nudge 是**每次 tool_result 都打一次的**。不是只在开头、也不是隔三差五；只要 plan 还没做完、还在进行中，它就一直打。模型的注意力一直被这条提醒拉回到「我的 plan 进度是怎样的」上面，忘记更新的概率就大大降低。

你知道吗——nudge 与 `<system-reminder>` ?

这种「在特定时机由 harness 主动往 context 里塞一条提醒」的机制，在现代 agent harness 里是常规武器，通称 **nudge**。Claude Code 里的 `<system-reminder>` 就是这一套思路——当 harness 检测到某些条件（tool_use 计数达到某个阈值、用户似乎 idle 了一段时间、某个关键文件被修改…），就会往 LLM 的下次 input 里插入一段系统级的提醒。

它既不是 user message 也不是 assistant message，是 **harness 层** 打进去的上下文补丁。本文开头你看到的 `<system-reminder>` 块就是这样来的。nudge 的核心认识是：**prompt engineering 是静态的，agent 的行为是动态的**——有些问题靠 prompt 写死治不了，只能靠 harness 在运行时针对当前状态主动干预。

补丁 A 治了 plan 的输入侧，补丁 B 治了 plan 的输出侧。加上开头那一件 `write_todos` 工具，Deep Research 这个场景下的 harness 就算搭完了——从单轮 ReAct，到多轮 Loop，再到 Plan-then-Act 加两个小补丁，一共就这点东西。

但同样一套骨架换一批工具会变成什么？下一章我们把镜头切到写代码的场景。



Agentara Underground 情报站 (2群)

每日投喂AI/Agent新闻，不定期推送热点深度研...

群名片

加入 Agentara Underground 情报站

加群获取更多线上资料，和 5,700+ 位 AI 爱好者一起学习 Prompt、Context Engineering、Harness Engineering 和 Skills。

6. 换个场景：Coding Agent

现在做一个小实验：骨架不动，把这三件工具换掉，看会发生什么。

6.1 换三件工具

System prompt 几乎完全别动，只添加一个 `workspace`，也就是当前的工作空间根目录：

代码块

```
1 <workspace>
2 <root path="/Users/henry/Desktop/academic-sourcing"/>
3 </root>
4 </workspace>
```

工具换成这三件，用 JSON schema 描述：

代码块

```
1  [
2    {
3      "name": "read_file",
4      "description": "Reads and returns the contents of a file at the specified
5      path. Use when you need to inspect or retrieve text from an existing file.",
6      "strict": true,
7      "parameters": {
8        "type": "object",
9        "required": [
10         "description",
11         "path"
12       ],
13       "properties": {
14         "description": {
15           "type": "string",
16           "description": "Explain why you need to execute this tool. This
17           property should always come as the first parameter."
18         },
19         "path": {
20           "type": "string",
21           "description": "The filesystem path of the file to read"
22         }
23       },
24       "additionalProperties": false
25     },
26     {
27       "name": "write_file",
28       "description": "Writes content to a file at the specified path, creating
29       or overwriting the file as needed. Use when you need to create a new file with
30       content. Do not use when you need to edit.",
31       "strict": true,
32       "parameters": {
33         "type": "object",
34         "required": ["description", "path", "content"],
35         "properties": {
36           "description": {
37             "type": "string",
38             "description": "Explain why you need to execute this tool"
39           },
40           "path": {
41             "type": "string",
42             "description": "The filesystem path of the file to write"
43           },
44           "content": {
45             "type": "string",
46             "description": "The full text content to write into the file"
47           }
48         }
49       }
50     }
51   ]
```

```

44     }
45 },
46 "additionalProperties": false
47 }
48 },
49 {
50     "name": "bash",
51     "description": "Executes a shell command in a bash environment and returns
the result. Use when you need to run terminal commands, scripts, or system
utilities. Do not use when a direct file read or write operation is
sufficient.",
52     "strict": true,
53     "parameters": {
54         "type": "object",
55         "required": ["description", "command"],
56         "properties": {
57             "description": {
58                 "type": "string",
59                 "description": "Explain why you need to execute this tool"
60             },
61             "command": {
62                 "type": "string",
63                 "description": "The bash command to execute"
64             }
65         },
66         "additionalProperties": false
67     }
68 }
69 ]

```

就三件。 `read_file` 读文件、 `write_file` 写文件、 `bash` 跑 shell。



你知道吗？ `description` 参数

有一个有趣的现象是，他们都有一个 `description` 参数，并且在 Claude Code 的工具参数说明里，都强调了要将其作为第一个参数输出。相信大家都想到了，这是为了让用户在 UI 上更好的理解为什么要执行这个步骤，也避免了看不懂命令行的尴尬。

6.2 看看它怎么修 bug

假设用户让它修一个测试失败的 bug，transcript 大致会长这样：

代码块

```
1 user: src/adder.py 里的 add() 有 bug, pytest 挂了，帮我修一下
```

```

2
3 assistant → bash("pytest tests/test_adder.py")
4 tool_result:
5     FAILED tests/test_adder.py::test_negative
6     assert add(-1, 1) == 0 # got -2
7
8 assistant → read_file("src/adder.py")
9 tool_result:
10    def add(a, b):
11        return a - b    # ← 一眼就看出来了
12
13 assistant → write_file(
14     "src/adder.py",
15     "def add(a, b):\n    return a + b\n"
16 )
17
18 assistant → bash("pytest tests/test_adder.py")
19 tool_result:
20     2 passed in 0.03s
21
22 assistant: 已修复。add() 原本写成了 a - b, 现在改回 a + b, 测试全通过。

```

完整的 code-change loop 跑完了——跑测试暴露问题、读源码定位问题、写文件改问题、再跑测试验证。整个过程用的还是 ReAct Loop 那一套：每一步都是 `think → tool_use → tool_result → think`，没有任何新的控制流。

画成图：



三件工具，一个 Loop，就足够跑一个修 bug 的完整回合。如果任务更复杂一点——比如加一个新 feature、跨几个文件——再把前面的 Plan-then-Act 套上去，`write_todos` 一下，执行流就和 Deep Research 完全同构了。

这就是这一章的 punchline——**harness 的骨架是通用的，工具的选型决定了 Agent 的形态**。换掉 `web_search` / `web_fetch`，加上 `read_file` / `write_file` / `bash`，Deep Research Agent 就变成了 Coding Agent。两个看起来完全不同的产品，底层跑的是同一套 Loop。



你知道吗——为什么 `write_file` 要有 `is_appending` ？

不是为了偷懒，是为了省 context。`write_file` 每次调用都要把完整内容放在 args 里传过去——一个 1000 行的文件，想在末尾追加一行日志，如果没有 append 模式，模型就得把完整的 1000 行先塞进 `content` 字段里重新传一遍。这 1000 行会出现在 `tool_use` 的

input 里，原封不动地进 context；下一轮还在 context 里，再下一轮还在。十几轮之后 context window 就被这几次无谓的重写占掉大半了。is_appending 开关的本质，是把「只增量修改」这个高频场景从「全量重写」里拆出来，避免模型每次都为了改一小段付出全量的 context 代价。

说到 context 代价——有些工具的返回值天生就大。bash 跑一次构建，stdout 几千行往上；read_file 一个大文件几百行起步；web_fetch 拉一整篇网页，一下子几万 token。这些东西如果原样全进 context，十几轮之后就会把窗口撑爆。

这是下一章要讲的事——Context Offloading。

7. Offloading：当 Context 装不下

详见 [Harness 101: Context Offloading 机制](#) 一文

上一章埋了一根引线——Coding Agent 的 bash、web_fetch、read_file 每一个都可能吐出几千 Token 的结果。一次 grep -rn 扫一个中型仓库，几千行日志；一次 web_fetch 抓一张 API 文档页面，几千 Token 的 HTML 正文。ReAct Loop 转着转着，几十轮下来，messages 历史里堆的几乎全是这些“吃一次就没用了”的 tool_result。

Context Window 不是无限的。200k Token 的模型，看起来很富裕，但跑一个稍长的 Coding Agent 任务，几十轮之后肉眼可见地逼近上限。到时候会发生什么？模型开始漏看前文、开始前后矛盾、甚至直接报错。

Harness 层给这个问题的第一张答卷，叫 Context Offloading。

你知道吗——什么是 Context Offloading?

Harness 层的一种机制：当某次 tool call 的参数或返回值体积超过阈值时，Harness 把原始数据写入本地磁盘，在 message 历史里只留一个路径占位符，换取 Context Window 的可回收性。它不是压缩、不是摘要，而是一次“搬家”——数据还在，只是不再常驻上下文。

7.1 一句话机制

Offloading 做的事非常朴素：tool_result 太大时，把原始内容写到本地磁盘，Context 里只留一个路径引用。Agent 下次需要时，再用 read_file 把它拉回来。

具体长什么样？一次 bash 调用吐出了 1k Token，Agent 看到的 tool_result 大致是这样：

代码块

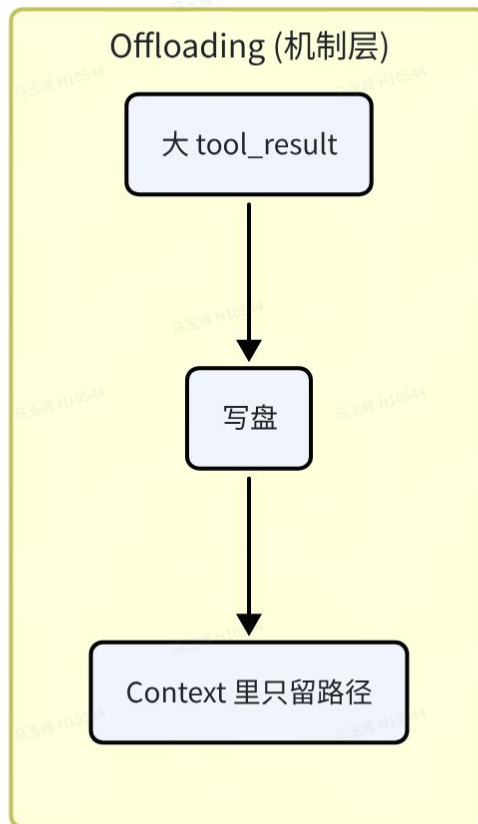
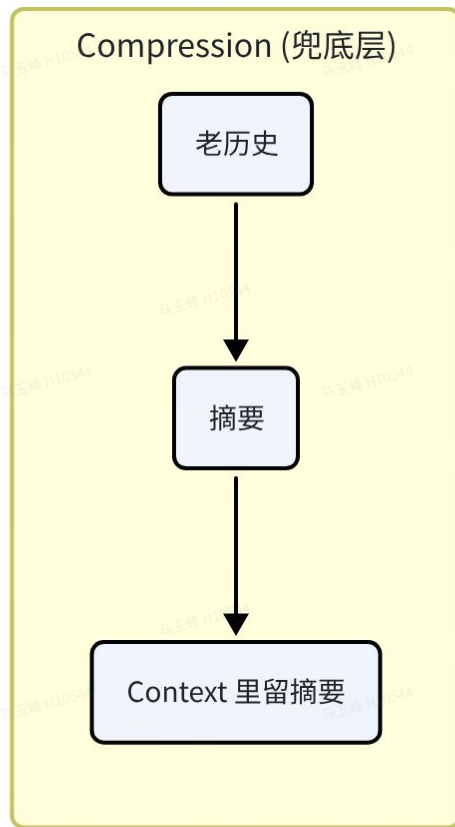
```
1  {
2    "role": "tool",
3    "tool_call_id": "call_042",
4    "content": "[OFFLOADED] saved to abc123.log (1024 tokens). Use read_file to
5    retrieve."
6  }
```

原始那 1k Token 没有进入 Agent 的 Context——它被 Harness 截在了返回的路上，搬到了磁盘。Context 里出现的是一个三十几 Token 的占位符。

从 Agent 自己的视角看，这件事甚至是无感的。它照常调用 `bash`、照常收 `tool_result`、照常决定"要不要 `read_file` 拉详细内容"。Offloading 是 Harness 偷偷做掉的事——它不引入新工具，也不改变 ReAct Loop 的任何一环，只是改写了 `tool_result` 的 `payload`。

7.2 Compression

这里容易和另一个概念混在一起——**Context Compression**。两者都是 Harness 在处理"Context 装不下"的问题，但机制截然不同。笔者用一张图把它们摆到一起：



差别有三层。

第一层是触发时机。 Offloading 发生在 tool_result 诞生的那一刻——Harness 看到返回体积超过阈值，当场就把它外置。Compression 则发生在 Context 快要爆掉的那一刻——等到占用率接近上限，

Harness 才回头翻历史、做摘要。

第二层是动作的有损与无损。 Offloading 是**无损可还原**的：原始内容一个字都没丢，只是换了存储位置，Agent 需要时 `read_file` 能原样拿回。Compression 则是**有损**的——`summarize` 把 10 轮对话压成 200 字摘要，丢掉的信息不可恢复。

第三层是抽象层次。 Offloading 活在**机制层**，处理单条 `tool_result` 的冷热分层；Compression 活在**兜底层**，处理整段历史的打包归档。一个像操作系统的分页调度，一个像把老日志塞进 tarball。

笔者喜欢这样总结：**Offloading 是冷热分层存储，Compression 是归档压缩。**前者优先——因为它无损、便宜；后者兜底——因为它有损、但能救命。

你知道吗——Offloading 很像操作系统的 `swap` ？

操作系统的 `swap` 机制：物理内存装不下时，把低频访问的内存页写到磁盘，在页表里只留一个引用；需要时再 swap-in 回来。单次 swap-in 看上去是亏的——多了一趟磁盘往返——但它让你能“用起”**远超物理内存的工作集**。Agent 的 Offloading 踩的是同一条杠杆：单次看 `tool_result` 下次还要 `read_file` 拉回来、账面吃亏；但从整条任务的时间线上看，绝大多数被 offload 的内容写出去就再也不会被读回——Context 的“有效容量”因此被放大好几倍。

7.3 点到为止

到这里，Offloading 的“是什么”和“和 Compression 有什么区别”已经讲完。但它真正的工程细节——什么时候触发、`args` 和 `results` 为什么走两条不同路径、触发阈值该设多少、与 Prompt Cache 的经济账怎么算——远比这一章复杂得多。那一层的完整拆解，笔者写在 [Harness 101: Context Offloading 机制](#) 里。想深入的读者可以去那篇继续读。

本章的任务只是把 Offloading 这个机制在 Harness 版图上的位置钉住——**当 `tool_result` 装不下时，Harness 用的是“搬家”而不是“瘦身”。**

解决了“装不下”的问题之后，还有一个更隐蔽的问题——这是 Skill 要处理的。



 **Agentara Underground 情报站（2群）** 外部

每日投喂AI/Agent新闻，不定期推送热点深度研…

群名片

加入 Agentara Underground 情报站

加群获取更多线上资料，和 5,100+ 位 AI 爱好者一起学习 Prompt、Context Engineering、Harness Engineering 和 Skills。

8. Skill: Harness 层的能力组织

前面几章讲的都是一个 Agent 在一次任务里怎么跑 Loop——ReAct 怎么转、Plan 什么时候插入、Nudge 怎么把它从岔路上拽回来、Offloading 怎么让 Context 装得下。这些机制解决的都是"单次任务的执行质量"。

但真实场景下，一个 Agent 往往挂着几十上百种能力——每个领域都有自己的最佳实践。一个 Coding Agent 可能同时要会：写 Python、改 Terraform、排查 Kubernetes、审 PR、生成周报、诊断告警……每一条都值得单独一份 SOP，每一条都有自己的模板、引用资料、陷阱清单。

把这些全塞 System Prompt 会怎样？一个最小 Skill 3k Token，五十个就是 **150k**，直接把主流模型的 Context Window 吃掉大半。即便硬塞得下，模型也会被信息噪声淹没——每一轮推理都要在一百条几乎无关的 SOP 之间分配注意力，命中率断崖式下跌。

Harness 层给"能力爆炸"这件事的答卷，叫 **Skill**。

8.1 一次装进去的只是目录

Skill 的机制朴素得出奇——每个 Skill 是磁盘上的一个文件夹，里面有一份 `SKILL.md`；Harness 启动时扫描所有 Skill 目录，只把每个 `SKILL.md` 的 `Frontmatter` (`name` + `description`) 拼成一份目录清单，注入 System Prompt 的 `<skills />` 节。正文留在磁盘上，不进 Context。看一眼 Claude Code 的 `<skill-system>` 片段——这是真实在线上跑着的那一套：

代码块

```
1 <skill-system>
2 <instruction>
3 You have access to skills that provide optimized workflows for specific tasks.
  Each skill contains best practices, frameworks, and references to additional
  resources.
4
5 **Progressive Loading Pattern:**
6 1. When a user query matches a skill's use case, immediately call `read_file`
  on the skill's main file using the path attribute provided in the skill tag
  below
7 2. If an explicit requested skill is provided in the system context, load that
  skill first even if the user message is short
8 3. Read and understand the skill's workflow and instructions
9 4. The skill file contains references to external resources under the same
  folder
10 5. Load referenced resources only when needed during execution
11 6. Follow the skill's instructions precisely
12 </instruction>
13 <skills>
```

```

14 <skill name="image-generation" path="/Users/henry/.agents/skills/image-
    generation/SKILL.md">
15 Use this skill when the user requests to generate, create, imagine, or
    visualize images ...
16 </skill>
17 <skill name="skill-creator" path="/Users/henry/.agents/skills/skill-
    creator/SKILL.md">
18 Create new skills, modify and improve existing skills, and measure skill
    performance ...
19 </skill>
20 </skills>
21 </skill-system>

```

这段 prompt 里最值得注意的不是它有什么——而是它**没有什么**。

没有新工具。 Harness 不需要专门为 Skill 造一个 `invoke_skill()` 或者 `load_skill()` 函数。<instruction> 里明明白白写着：immediately call read_file on the skill's main file using the path attribute。模型判断要用某个 Skill 时，发出的是一条普通的 `read_file` 调用——Coding Agent 本来就有这个工具。**Skill 复用了文件系统这个抽象，一个新工具都没加。**

没有正文。 这里每个 <skill> 标签里只有几十字的 `description`。SKILL.md 里真正那几千字的正文躺在 `path` 指向的磁盘位置，等着被需要的那一轮用 `read_file` 拉进来——这就是 Progressive Disclosure（渐进式披露）在 Harness 层的表现：**metadata 常驻 Context，body 按需 load。**

8.2 Harness 启动时做的那件事

一份伪代码把 Harness 启动时的动作摆出来：

代码块

```

1 # 伪代码：Harness 启动时拼 <skills /> 节
2 def build_skills_section(skills_root: Path) -> str:
3     skills = []
4     for skill_dir in scan_dirs(skills_root):
5         skill_md = skill_dir / "SKILL.md"
6         if not skill_md.exists():
7             continue
8         fm = parse_frontmatter(skill_md) # 只读 YAML 头，不读正文
9         skills.append(
10             f'<skill name="{fm.name}" path="{skill_md}">\n'
11             f'{fm.description}\n'
12             f'</skill>'
13         )
14     return "<skills>\n" + "\n".join(skills) + "\n</skills>"

```

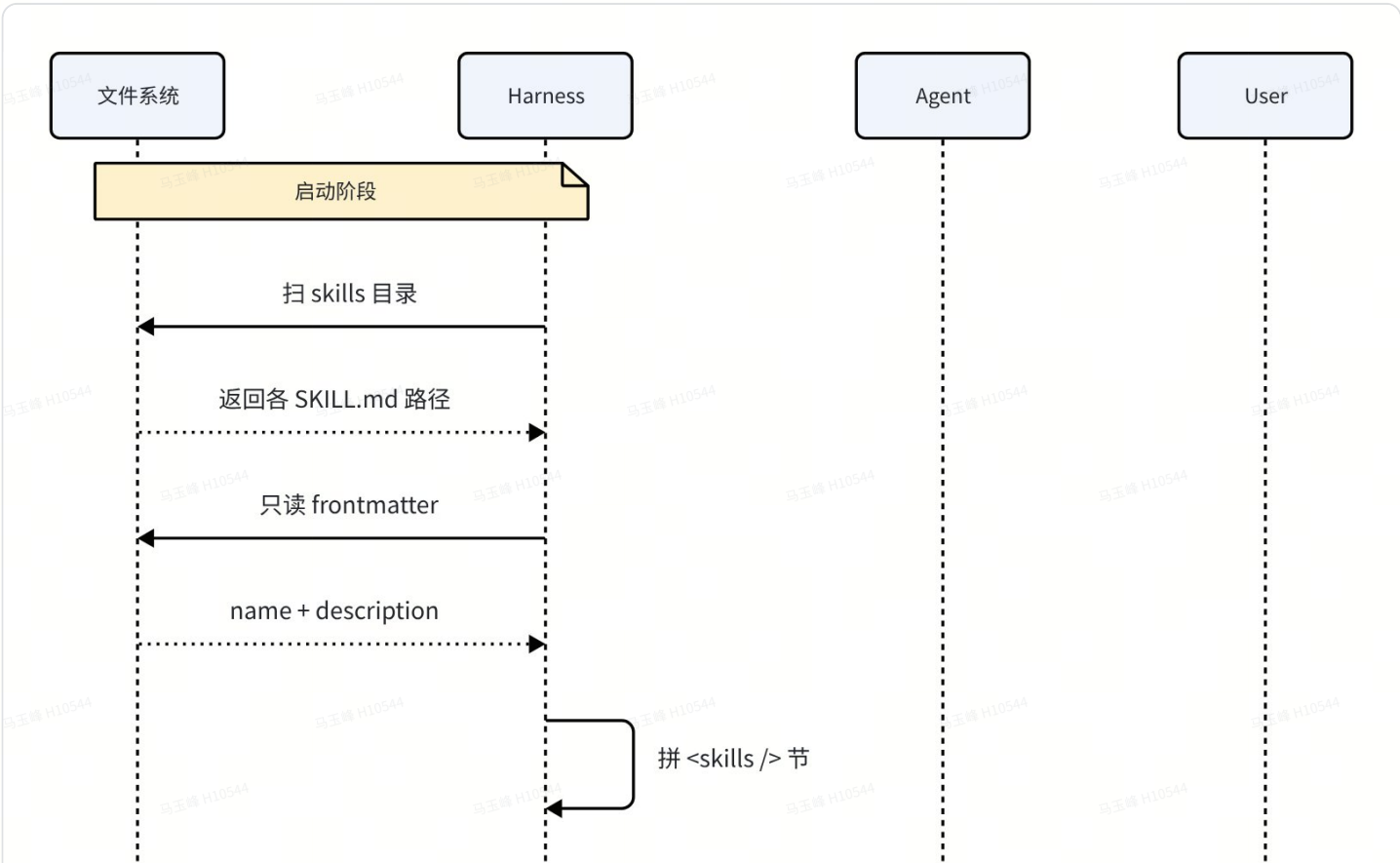
注意 `parse_frontmatter` 这一步——它只吃文件头那段 `---` 包起来的 YAML, `SKILL.md` 的正文 (body) 从头到尾没被读进来。一个装了 100 个 Skill 的 Agent, 启动时 Harness 总共读的数据量, 可能还不如它随手一次 `read_file` 读一份日志。

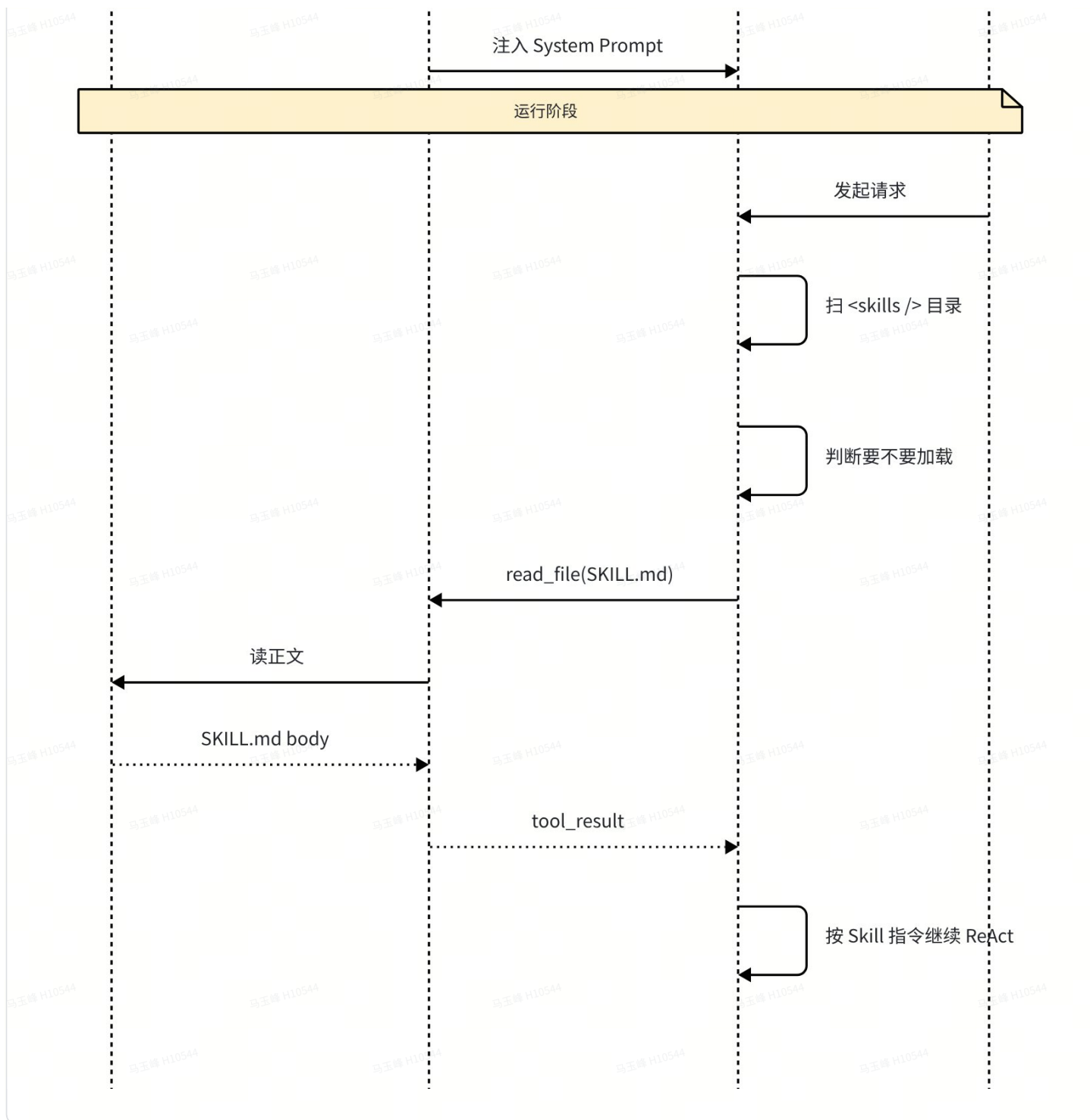
8.3 为什么不用 BM25 或 Embedding 做召回

读到这里, 工程直觉强的读者会反问: 挑 Skill 不就是个检索问题吗? "哦, 这是 RAG / Embedding 检索吧?" 答案是否定的。向量召回依赖语义相似度——把用户请求编码成向量, 与 Skill 描述的向量做近邻匹配, 属于被动匹配; 对于抽象或换了说法的请求很容易漏召回, 更致命的是它一次就结束, 串不起多个工具。Skill 触发依赖 Agent 的 `ReAct` (Reasoning + Acting) 式自主决策——模型边推理边挑工具, 理解的是意图而不是字面。举个具体的: 一句"帮我排查刚才那个线上报警", 理想解法是连锁调用三个看似无关的 Skill——`log-grep` (检索时间窗内的异常日志)、`changelog-reader` (读最近一次发布说明)、`incident-writer` (按公司格式写事故复盘)。这三者的 `description` 在向量空间里和"排查报警"这个 query 的距离完全不一样: 第一个勉强沾边, 后两个几乎毫无相似度, 一次性近邻检索最多交出 `log-grep` 一个孤零零的结果。而在 `ReAct` 循环里, Agent 会沿着自己的推理链一步步点亮它们——看日志发现错误时间点 → 想到查最近发布 → 定位后写复盘。这种"看似无关的 Skill 被连锁点燃"的能力, 是任何 Embedding 召回都做不到的, 也是为什么 `description` 字段写得好不好至关重要——它是 Skill 等着被 Agent 在推理链条里识别并调用的"名片", 不是一个被动等待匹配的向量。

8.4 整套流程的时序

把启动、加载、触发、使用画在一起:





时序图里唯一值得盯一眼的是中间那条 `read_file` 调用——它用的是 Coding Agent 早就有的通用工具，不是任何新抽象。Harness 对 Skill 的支持，在工具维度上是零增量的。



你知道吗—— `<skills />` 为什么放在 System Prompt 里而不是做成 tool 返回？

想象另一种设计：提供一个 `list_skills()` 工具，Agent 想找能力时主动调用、拿到目录、再挑。听起来干净，但有个致命问题——tool 返回只在 Agent 主动调用它的那一轮才出现在 Context 里。Agent 不一定每一轮都“想起要查一下可用 Skill”——很多时候它会基于已有 Context 直接开干，错过本该触发的 Skill。相反，`<skills />` 写在 System Prompt 里意味着它每一轮推理都在场——Agent 不需要“想起来去查”，目录本来就在眼前。这是“常

驻 metadata"与"按需 body"分层的底层动机：什么东西该让模型每一轮都看到，什么东西该等到用时再展开，这条线在 Skill 的设计里被划得非常干净。

思考题：但当用户的 Skills 超过 100 个、1,000 个，还能将所有 Skills 的 Frontmatter 信息放在上下文里吗？

8.5 把 Skill 放回 Harness 版图

Skill 不改 ReAct Loop、不加新工具、不动 Context Offloading 的任何一层——它只是在 System Prompt 里多塞一段 `<skills />`，然后把真正的正文推迟到触发那一刻再用 `read_file` 拉进来。

但就是这个小小的"推迟"动作，让 Harness 层支持的 Agent 从"能干一件事"跃迁到"能干几十上百件事"。Skill 是 Harness 给**能力增长**这件事的答卷。

至于怎么写好一个 Skill、`description` 怎么措辞、`Progressive Disclosure` 的三层结构如何各司其职、它和 Tool / Prompt / Script 的象限地图——这些本章不再重复。笔者在 [Skill 101：前传](#) 里已经写了一篇完整的铺垫，想深入的读者可以从那里接着读。

本章的任务到此为止：把 Skill 这一机制在 Harness 版图上的位置标清楚。接下来该收尾了。

9. 结语

回头看这一整篇文章，从 ReAct Loop 出发，一路讲了 Plan-then-Act、Nudge、Coding Agent 的工具三件套、Context Offloading、Skill——值得把主线重新拎一下。

Harness 不是一个黑盒中间件。它是由**一组正交机制**叠起来的抽象层。

ReAct Loop 管"怎么转"——它决定了 Agent 最基本的运转节拍：一步一步 Reason → Act → Observe，不贪快也不提前下判断。

Plan-then-Act 管"要不要先想再做"——它在 Loop 外面套了一层拐弯：开工前先停下来画一张施工图，拿到用户点头再动手。

Nudge 管"怎么被提醒"——它是 Harness 轻轻拍 Agent 肩膀的那只手，在后者快要走偏时递一张便签回来。

Offloading 管"记不下就先搬出去"——它让 Context Window 看起来像是无限的，代价只是一次磁盘往返。

Skill 管"能力太多怎么分层放"——它把几十上百条 SOP 推到磁盘上，只让"目录清单"常驻 Context，用时再翻书。

把这五层按顺序读下来会发现——**每一层都是对上一层暴露出来的问题的局部回应**。ReAct Loop 本身很朴素，但朴素的 Loop 跑起来就会暴露"Agent 容易横冲直撞"的问题——于是有了 Plan；Plan 解决了主动规划，但 Agent 仍然会在长程任务里"忘事"——于是有了 Nudge；Nudge 解决了提醒，但 tool_result 还是会把 Context 撑爆——于是有了 Offloading；Offloading 解决了单条数据的装载，但 System Prompt 本身会被能力堆爆——于是有了 Skill。

一层一层往上摞，每一层都**只做一件事**，每一层都**假设下面几层已经稳定**。

笔者越看这幅图，越觉得它和两个老朋友的味道像——**TCP/IP 的分层，操作系统的子系统拆分**。好的 Harness 设计不是一次把所有层想清楚再动手，而是**允许每一层被单独调整和替换**。今天你改了 Offloading 的阈值策略，不会伤到 Skill 的加载机制；明天你换了一种 Nudge 的触发规则，不会动到 ReAct 的基本节拍。每一层都留了独立的观察点、独立的配置面板、独立的版本号——这是真正让一个 Harness **能活下去、长起来的骨架**。

Harness 不是在一张白纸上一次画完的蓝图。它是被每一个"暴露出来的问题"**倒逼**出来的分层——一层解决一类问题，一层预留一个未来。

这个系列只是开了个头。后续《Harness 101》每一个机制都值得单独拆一篇：

- 《**Harness 101: Context Offloading 机制**》已经发了——拆的是"两个机制、args/results 非对称、阈值的经济学"
- 《**Skill 101: 前传**》也已经发了——拆的是"Skill 的四个核心要素、Progressive Disclosure、它的能力边界"

后面可能继续写的方向还有——**多 Agent 协同与 Sub-Agent 抽象**（一个 Harness 怎么调度好几个 Agent）、**Compression 的策略光谱**（当 Offloading 都兜不住了，摘要层怎么兜）、**Tool Result 的结构化与 streaming**（当单条返回大到必须流式，Harness 怎么办）。

每一篇都会是一次局部深挖。有兴趣的读者，我们下一篇见。

领取教程材料



课程材料.zip
63.82KB



[同步块](#) [该内容不支持导出查看]

加入 Agentara Underground 情报站

加群获取更多线上资料，和 6,000+ 位 AI 爱好者一起学习 Prompt、Context Engineering、



Agentara Underground 情报站 (2群)

外部

每日投喂AI/Agent新闻，不定期推送热点深度研…

群名片

Harness Engineering 和 Skills。